arXiv:2409.12951v2 [cs.LG] 1 Feb 2025

Geometric Interpretation of Layer Normalization and a Comparative Analysis with RMSNorm

Akshat Gupta, Atahan Ozdemir, Gopala Anumanchipalli
UC Berkeley
{akshat.gupta, atahanozdemir, gopala}@berkeley.edu

Abstract

This paper presents a novel geometric interpretation of LayerNorm and explores how LayerNorm influences the norm and orientation of hidden vectors in the representation space. With these geometric insights, we prepare the foundation for comparing LayerNorm with RMSNorm. We show that the definition of LayerNorm is innately linked to the uniform vector, defined as $\mathbf{1} = [1, 1, 1, 1, \cdots, 1]^T \in \mathbb{R}^d$. We then show that the standardization step in LayerNorm can be understood in three simple steps: (i) remove the component of a vector along the uniform vector, (ii) normalize the remaining vector, and (iii) scale the resultant vector by $\sqrt{d}$, where d is the dimensionality of the representation space. We also provide additional insights into how LayerNorm operates at inference time. Finally, we compare the hidden representations of LayerNorm-based LLMs with models trained using RMSNorm and show that all LLMs naturally operate orthogonal to the uniform vector at inference time, that is, on average they do not have a component along the uniform vector during inference. This presents the first mechanistic evidence that removing the component along the uniform vector in LayerNorm is a redundant step. These results advocate for using RMSNorm over LayerNorm which is also more computationally efficient.

1 Introduction

The transformer architecture (Vaswani et al., 2017) has been the cornerstone of most recent advances in artificial intelligence and has rapidly become the architecture of choice in natural language processing, computer vision, speech, and various other domains. Layer normalization, a crucial yet often overlooked component of the transformer architecture, plays an integral role in stabilizing the training process in transformer-based models. Introduced by Ba et al. (2016), layer normalization standardizes the features at each layer, adjusting and scaling the activations to have zero mean and unit variance within a vector. This normalization is performed independently for each hidden vector in contrast to batch normalization (Ioffe and Szegedy, 2015), which relies on statistics (mean and variance) from a batch of data points. Layer normalization is especially effective when working with long sequences of variable lengths. While the benefits of layer normalization, such as improved training speed and better convergence, are well documented (Ba et al., 2016; Xu et al., 2019; Zhang and Sennrich, 2019; Jiang et al., 2024), its specific effects on the hidden vectors within a model and the global properties of the resulting representations remain surprisingly underexplored.

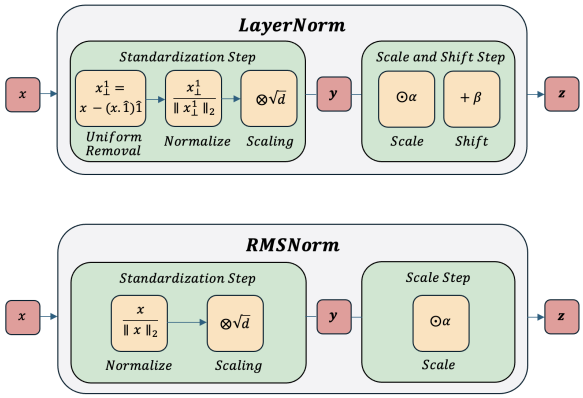
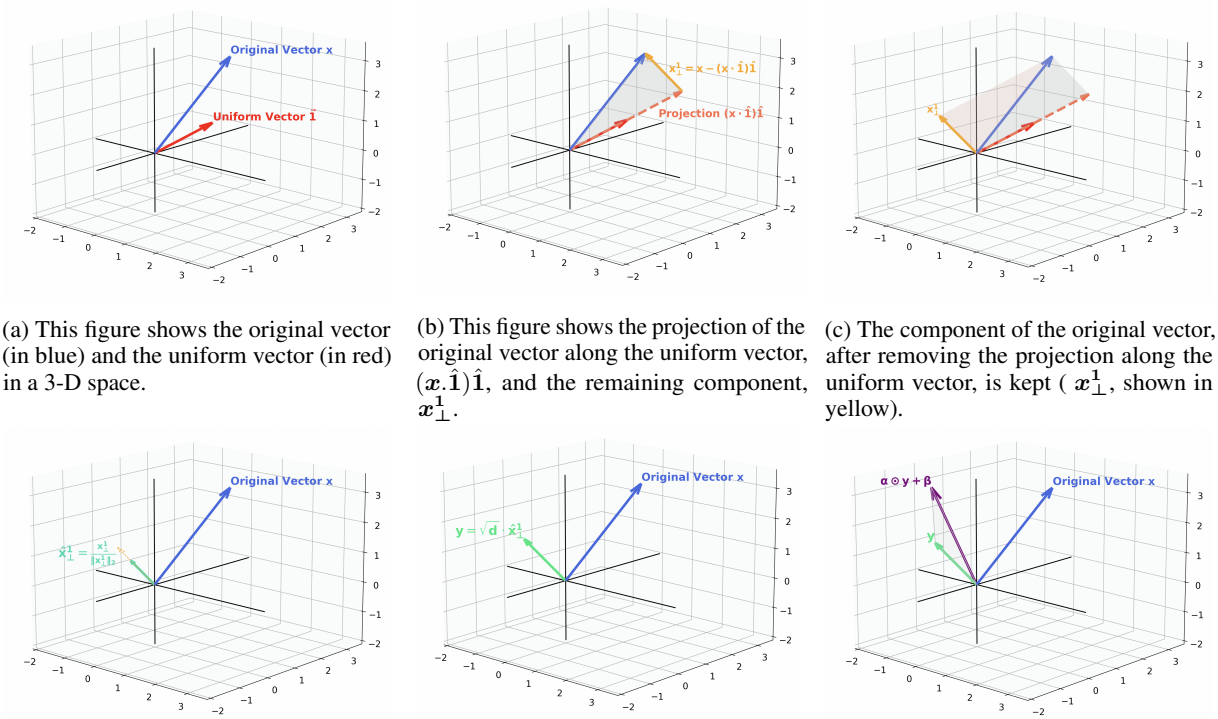


Figure 1: A diagrammatic explanation of LayerNorm and RMSNorm.

In this paper, we first discuss the global effects of layer normalization on a vector. The conventional explanation of the LayerNorm operation is usually as follows: *standardize each vector by subtracting the mean of its elements, divide by the standard deviation*. While this is an accurate procedural definition, we ask a more global question: **How does the LayerNorm operation transform a vector in representation space?**

We present a novel interpretation of LayerNorm and show that it can be understood in three steps:



(a) This figure shows the original vector (in blue) and the uniform vector (in red) in a 3-D space.

(b) This figure shows the projection of the original vector along the uniform vector, $(\boldsymbol{x}.\hat{\mathbf{1}})\hat{\mathbf{1}}$, and the remaining component, $\boldsymbol{x}^{\mathbf{1}}_{\perp}$.

(c) The component of the original vector, after removing the projection along the uniform vector, is kept ($\boldsymbol{x}^{\mathbf{1}}_{\perp}$, shown in yellow).

(d) Normalizing perpendicular component to the uniform vector ($\boldsymbol{x}^{\mathbf{1}}_{\perp}$) to unit norm.

(e) Scale of resultant vector by $\sqrt{d}$, where d is the dimensionality of the representation space.

(f) Finally, the scale-and-shift step, which scales and shifts the resulting vector according to learnt parameters.

Figure 2: Visualization of LayerNorm operation on a random original vector

(i) throw away the component of the given vector along $\mathbf{1} = [1, 1, 1, 1, \cdots, 1]^T \in \mathbb{R}^d$, (ii) normalize the resultant vector; and (iii) scale the resulting vector by $\sqrt{d}$, where d is the dimensionality of the vector space. This process is illustrated in Figure 2. This operation throws away the information along $\mathbf{1} = [1, 1, 1, 1, \cdots 1]^T$, which we call the uniform vector, indicating that the information along the uniform vector may not be important.

We then present the property of "irreversibility," where we show that the information lost in the LayerNorm process cannot be recovered. These results naturally lead to discussion about the importance of information along the uniform vector, which is removed by LayerNorm irreversibly, and its comparison with RMSNorm (Zhang and Sennrich, 2019), a variant of LayerNorm that doesn't remove the component along the uniform vector and is used to train the latest Llama models (Touvron et al., 2023a,b; Dubey et al., 2024). Figure 1 shows the difference between the two normalization methods. To empirically justify removing the components of hidden vectors along the uniform vectors, the hidden representations should have non-trivial components along the uniform vector.

We empirically show that both LayerNorm-based models naturally produce hidden representations orthogonal to the uniform vector during inference time, thus rendering the removal of the uniform vector in LayerNorm inconsequential in practice. This is true even for RMSNorm-based models, where hidden vectors on average operate othogonal to the uniform vector. With these results, we provide both theoretical and empirical justifications for the redundancy of removing the component of hidden representations along the uniform vector, thus supporting the usage of RMSNorm over LayerNorm, which is also more computationally efficient.

To summarize, in this paper, we make the following contributions:

1. Provide a simple geometric interpretation of the layer normalization process that offers an intuitive explanation of how LayerNorm transforms a vector in representation space.

2. Present LayerNorm in action with experiments depicting its importance in stabilizing the residual stream vectors.

3. Show that removing the component along the uniform vector is a redundant step in LayerNorm at inference time. These findings ad-

vocate for the adoption of RMSNorm over LayerNorm in LLMs.

2 Re-Introducing Layer Normalization

Let $\boldsymbol{x} \in \mathbb{R}^d$ represent an intermediate hidden representation in a transformer-based model, on which the LayerNorm operation is applied. Then the following steps summarize the layer normalization operation:

1. Calculate the mean, $\mu = \frac{1}{d}\sum_{i=1}^{d} x_i$, and standard deviation, $\sigma = \sqrt{\frac{1}{d}\sum_{i=1}^{d}(x_i - \mu)^2}$, where x_i are the components of $\boldsymbol{x}$

2. Standardize the components of the hidden vector $\boldsymbol{x}$ to get $\boldsymbol{y}$, such that $y_i = \frac{x_i - \mu}{\sigma}, \forall x_i \in \boldsymbol{x}$

3. Scale and shift to get $\boldsymbol{z} = \boldsymbol{\alpha} \odot \boldsymbol{y} + \boldsymbol{\beta}$, where $\odot$ represents element-wise multiplication of two vectors, and $\boldsymbol{\alpha}, \boldsymbol{\beta}$ are scaling and shifting vector parameters learned during training.

In this paper, we refer to the combination of step-1 and step-2 as the *"standardization step"*, whereas step-3 is referred to as the *"scale-and-shift"* step. The removal of the mean of components from each component in step-1 is referred to as the *"mean subtraction"* step in this paper. For an overview of the computations happening within a decoder layer in modern LLMs, we refer the reader to section A.1.

While the original definition of LayerNorm gives us an operational description, it tells us very little about the global properties of the resulting vectors. For example, with the steps described above, we get very little insight into the norm or the orientation of the standardized vector ($\boldsymbol{y}$). All we know is that the components of the vector $\boldsymbol{y}$ are standardized to have zero mean and unit variance within the vector. But how is this standardized vector oriented compared to the original vector and what are its norms? Such an explanation was also absent in the original formulation of layer normalization (Ba et al., 2016).

Initial explorations into the geometry of layer normalization by Brody et al. (2023) demonstrated that $\boldsymbol{y}$ maintains a norm of $\sqrt{d}$ and is oriented orthogonal to the constant vector $\mathbf{1} = [1, 1, 1, 1, \cdots, 1]^T \in \mathbb{R}^d$, where d represents the dimensionality of the vector space of hidden vectors. Furthermore, a popular variant of LayerNorm, known as RMSNorm (Zhang and Sennrich, 2019), which omits the "mean subtraction" in the standardization step, has come out as a viable alternative to LayerNorm. In this paper, we expand upon these foundational studies by presenting a novel interpretation of the global effects of LayerNorm on a vector, which we argue provides a more intuitive and informative description of how LayerNorm modifies a hidden vector in representation space. Additionally, we analyze the representations of models using RMSNorm, showing that despite omitting the "mean subtraction step", RMSNorm produces hidden representations with similar orientations.

To understand the global effect of LayerNorm on a vector $\boldsymbol{x}$, we need to represent the standardization steps in terms of the vector $\boldsymbol{x}$ and not its components. A neat way to write the mean of the components of a vector $\boldsymbol{x}$ is:

$$\mu = \frac{1}{d}\mathbf{1}^T\boldsymbol{x} \tag{1}$$

where $\mathbf{1} = [1, 1, 1, 1, \cdots 1]^T$ such that $\mathbf{1} \in \mathbb{R}^d$, which we refer to as the *uniform vector* in this paper. Using the scalar mean value calculated above, we define the *mean vector* as

$$\boldsymbol{\mu} = \mu\mathbf{1} \tag{2}$$

where μ is a scalar mean as calculated in equation 1. Thus, the mean vector is a vector with each component equal to the mean of the components of vector $\boldsymbol{x}$. Using this, we can also rewrite the standard deviation of the components of $\boldsymbol{x}$ as:

$$\sigma = \sqrt{\frac{1}{d}(\boldsymbol{x} - \boldsymbol{\mu})^T(\boldsymbol{x} - \boldsymbol{\mu})} \tag{3}$$

Once we do this, we can now write the standardize step in the LayerNorm algorithm (step-2) in vector form as follows:

$$\boldsymbol{y} = \frac{1}{\sigma}(\boldsymbol{x} - \boldsymbol{\mu}) \tag{4}$$

In the above equation, we are assuming that the standard deviation is non-zero. As can be seen in equation 3, the standard deviation is zero only when $\boldsymbol{x} = \boldsymbol{\mu}$, which happens when all components of the hidden vector are equal. In practice, the authors of LayerNorm add an error term in the denominator to prevent this from happening. In the discussion that follows, we will assume that $\boldsymbol{x} \neq \boldsymbol{\mu}$, which is the same as saying that all components of $\boldsymbol{x}$ are not equal. This assumption does not lead to any loss of generality, as will be evident in later